

Chapter 1

The Uniqueness of Software Quality Assurance

The Software Quality Challenge

- This chapter is essentially about two major topics:
 - The uniqueness of software quality assurance
 - The environments for which SQA methods are developed.

The Uniqueness of Software Quality Assurance

Introduction

- Why study Quality Assurance and Testing?
- With all the methodology wars, numerous processes, huge number of tools to assist in software development, why this **separate topic**?
- methodology wars: XP, Scrum, UP, hybrids...
- What makes SQA important that it deserves so much attention?
- SQA is a key required course in software engineering curricula. Why??

Major Differences between Software Products and Industrial Products

- **1. High complexity of the software product**
 - The potential ways in which a software product can be used with different data / data paths reflecting different incoming data is almost infinite.
 - Manner in which industrial products can be used are usually much more succinctly defined.
 - Parameters for use are usually clearly spelled out.
 - Process for development clearly defined; prototype, ...
 - Think about software:
 - every loop with different values of data reflects a different opportunity to see software fail.
 - In truth, the number of paths through a non-trivial software product is infinite.

Differences between Software Products and Industrial Products

- **2. Invisibility of the software product**

- In an industrial product, missing parts are obvious.
 - Something missing? Easily identified. Clear 'spot' for it.
 - Parts missing easily identifiable too.
- Not so in software products.
 - May not be noticeable for years – if at all!
 - Cite: phantom paths product at AFDSDC!
 - Parts may have never been in the software ever!

Differences between Software Products and Industrial Products

- Consider: **Product Development and Production Process from three perspectives:**
- 1. For **Industrial Products**:
 - **Product Development** –
 - Designers and QA people check / test the prototype for defects
 - **Product Production Planning** –
 - Here, production process and tools are designed and prepared.
 - May require special production line to be designed and built
 - Lots of opportunities to check for defects that escaped reviews and tests conducted during development
 - **Manufacturing** –
 - **QA** procedures are applied to **detect failures** of the products themselves during **manufacturing**.
 - **Can be corrected by change** in the design or in production tools and this will change the way products are manufactured in the future.

Differences between Software Products and Industrial Products

- Will see that:
 - Comparing industrial products with software products we see that the **only phase** when defects can be corrected in software products is really in the **product development phase**.
- Let's look at the same three activities
 - (product development, product production planning, and manufacturing)
- for Software Products:

Differences between Software Products and Industrial Products

- Consider: **Product Development and Production Process from three perspectives**
- 2. For **Software Products**:
- **Product Development** – Best Chance to Detect Errors!
 - Here, we look for inherent product defects and hope to arrive at an acceptable prototype.
- **Product Production Planning**
 - This phase is not required for software production process.
 - Copies are simply reproduced / printed automatically....
 - Numbers of copies of no consequence.
 - Defects in one copy are found in additional copies. End of story!
- **Manufacturing**
 - Manufacturing **limited** to copying product / printing copies of manuals.
 - Chances for detect defects here is quite limited.

Only Chance to Discover Defects:

- **Best chance to really detect defects occurs during the software development process itself!**
- “The need for special tools and methods for the software industry is reflected in the professional publications as well in special standards devoted to SQA, such as ISO 9000-3, “Guidelines for the application of ISO 9001 to the development, supply, and maintenance of software.”
- Another: ISO 9004-2: “Quality Management and Quality Systems Elements: Guidelines for the Services.”
- The characteristics of software –
 - complexity,
 - invisibility, and cu
 - limited opportunity to detect bugs
- has led to the **development** of the ISO Guidelines and an acute **awareness** of the importance and essential SQA methodology.

The Environment for which SQA Methods are Developed

Frame 1.2 Summary of the main characteristics of SQA environment

1. Being contracted
2. Subjection to customer–supplier relationship
3. Requirement for teamwork
4. Need for cooperation and coordination with other development teams
5. Need for interfaces with other software systems
6. Need to continue carrying out a project while the team changes
7. Need to continue maintaining the software system for years

The Environment for which SQA Methods are Developed

- Let's look at the **environment** for professional software development and maintenance
- Main Characteristics: (We will look at each of these in turn)
 - 1. Contractual Conditions
 - 2. Subjection to Customer – Supplier Relationship
 - 3. Required Teamwork
 - 4. Cooperation and and coordination with other teams.
 - 5. Interfaces with other software systems
 - 6. The need to continue carrying out a project despite team member changes
 - 7. The need to continue carrying out software maintenance for an extended period.
- These are ALL activities where SQA in various forms is critical to successful software engineering work

The Environment for which SQA Methods are Developed

1. Contractual Conditions

- These include functional requirements, the project budget, and the project timetable.
- These are the biggies! Contracts are huge!!! (as are the other parameters)
- Who has responsibility for what...we will see.
- Delivering software on time, within budget that meets or exceeds the functional requirements constitutes the thrust and legal elementsof contracts.

The Environment for which SQA Methods are Developed

- 2. Subjection to Customer-Supplier Relationship
 - We must realize that the customer drives the process in many cases – submitting changes, evaluating deliverables, acceptance testing, accepting deployments, approving the deliverables
 - This relationship must be generally good, but can be horribly complicated!!
 - This is the relationship that is critical when software is developed by software professionals.
 - Many war stories available!

The Environment for which SQA Methods are Developed

- 3. Required Teamwork
 - Three very motivating factors for teams vice done individually:
 - Timetable requirements – team members work together
 - Have a variety of specializations – Discuss. Always necessary? Where? When?
 - Mutual support and review to enhance product quality

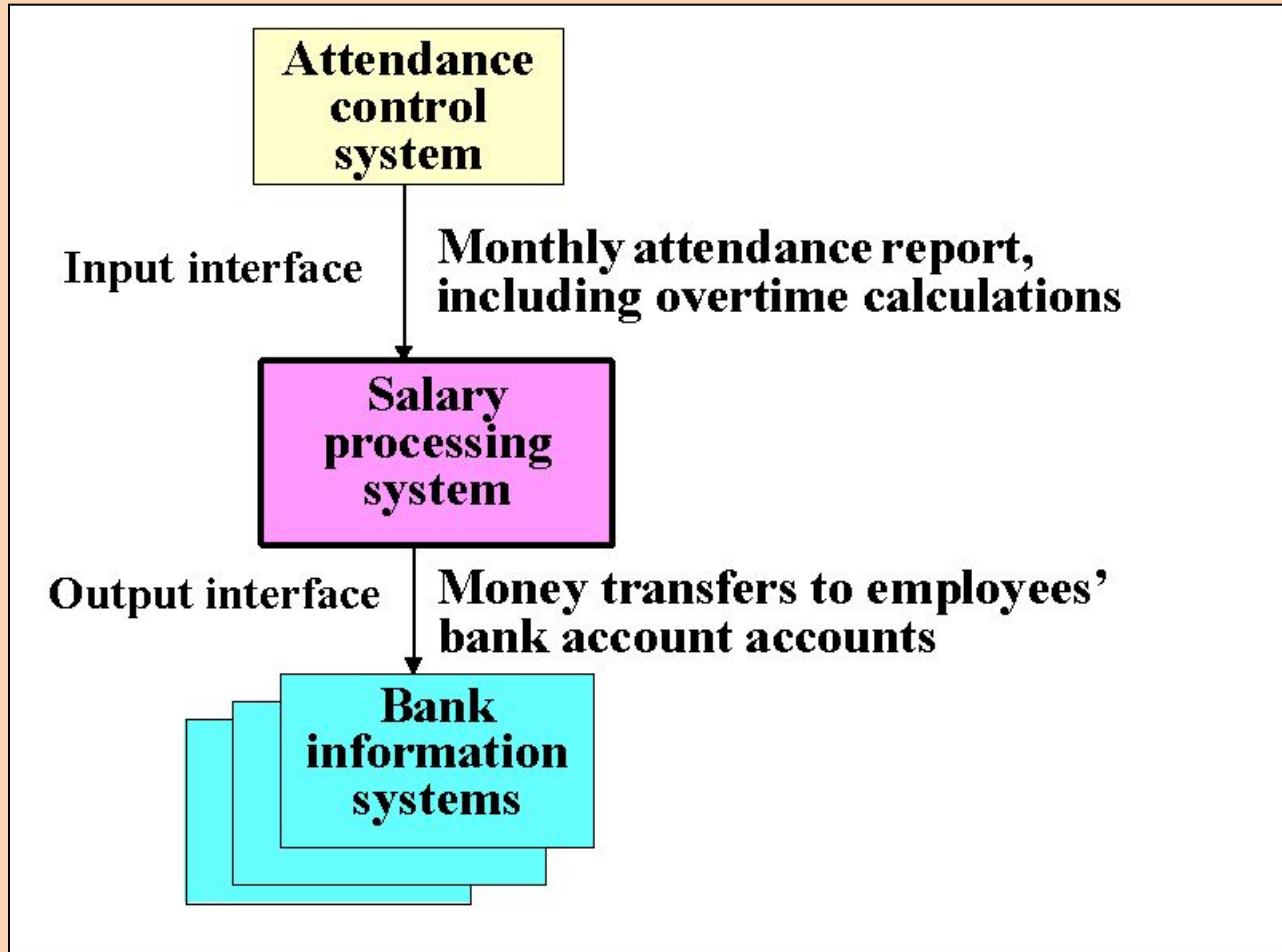
The Environment for which SQA Methods are Developed

- 4. Cooperating and coordination with other software teams
 - In a large software development organization, there are many teams and much development that requires coordination and compatibility issues
 - Expertise may exist in another team.
 - Can you borrow that person? That expertise?
 - Software development may outsourced in part.
 - Other teams may have developed similar software for the client and can offer tremendous help – perhaps...
 - Conflicts
 - Escalation!

The Environment for which SQA Methods are Developed

- 5. Interfaces with Other Systems
 - A Course Registration System – may well interface with a Class Scheduling System and perhaps a Billing System.
 - Oftentimes outputs from one system are inputs to another and vice versa!
 - A course registration system may need to ‘interface’ with an existing Billing system with different file / database formats, and more.
 - Sometimes outputs from one system are inputs to **several** others
 - Or, outputs from some system update master files / databases that are processed by other systems.

The Environment for which SQA Methods are Developed



The Environment for which SQA Methods are Developed

- 6. The Need to continue Carrying out a Project despite Team Member Changes
 - Very commonplace! Count on it!
 - “The show must go on”
 - Fred Brooks: Adding people to a late project makes it later.
 - Fred Brooks: Communications with new members
 - Fred Brooks: Getting new people up to speed.
 - Team members leave, are hired, fired, take unexpected vacations, transferred within the company, and more.
 - Maddening truism, but the development must continue.
 - You can count on disruption!

The Environment for which SQA Methods are Developed

- 7. The Need to Continue Carrying out Software Maintenance for an Extended Period
 - Software is developed to run for years.
 - This is what affects the company's bottom line – not software development, where the company is totally in the 'red.'
 - Maintenance is good and is where software development corporations make their money.
 - Remember, when developing software, company is in the 'red.'
 - Takes some time after deployment for transition into the black and make revenue.

Internal Software Development

- Not terribly different from external clients.
- But in-house development normally eschews formal contracts and/or formal customer/supplier relationships.
- Much in-house development or **upgrading** of in-house software is **typical**.
- The relationships between 'internal' customers and development varies greatly 'when measured by a formal-informal scale.'
- Some managers claim that the closer the relationships to the formal form, the greater the probability for project success.

Homework – Chapter 1

- You are to answer the following question in essay format and submit to me via Assignment 1.
- Question 1.3, p12
- Due: As stated in class room.

Discussion Forum

- One will lead a discussion and prepare materials to fully talk about the following questions in our next class:
- Present Brief Review of Chapter (3 minutes)
- Topics for discussion :
 - Question 1.1
 - Question 1.3, and
 - Question 1.5